DSA Lab – Experiments 6 to 9 (CMD Exam Edition)

Contents: 12 C programs, each a standalone file. Every program was compiled with `gcc -Wall -Wextra -std=c99 -pedantic` with **zero warnings**. Every transcript below comes from a real run with the exact input shown.

How to read the CMD blocks

What you see	Meaning
`C:\DSA>gcc ...` / `C:\DSA>expXX.exe`	**Command** you type at the prompt
Text after `Enter ...:` on the same line, and the matrix rows	**Input** you type (also listed under "You type" after each block)
Everything else	**Output** printed by the program

One-time setup in CMD

```
C:\>mkdir C:\DSA
C:\>cd C:\DSA
C:\DSA>notepad exp6a.c
```

Shared inputs (learn these once, reuse them):

- **Graph G4** (Exp 6a, 6b): `0 4 1 999 / 4 0 2 5 / 1 2 0 8 / 999 5 8 0`
- **Graph M4** (Exp 7a, 7b): `0 2 3 0 / 2 0 1 4 / 3 1 0 5 / 0 4 5 0`
- **Graph U5** (Exp 8a, 8b): `01100 / 10010 / 10001 / 01000 / 00100`, start at 0
- **Array** (Exp 9a–9d): `5 3 4 1 2`, which sorts to `1 2 3 4 5`
- **Keys** (Exp 9e, 9f): `23 43 13 27 37` with table size 10

EXPERIMENT 6 – Shortest Path Algorithms

6(a) Dijkstra's Algorithm – Single Source Shortest Path

Aim: To find the shortest distance from a source vertex to all other vertices using Dijkstra's algorithm.

Algorithm

1. Read the number of vertices `n` and the cost matrix (999 = no edge).
2. Read the source vertex `src`.
3. Set `dist[i] = cost[src][i]`, `dist[src] = 0`, and mark `src` visited.
4. Repeat `n-1` times: choose the unvisited vertex `u` with the smallest `dist`.
5. Mark `u` visited.
6. For every unvisited `v`: if `dist[u] + cost[u][v] < dist[v]`, update `dist[v]`.
7. Print `dist[]`. Stop.

Program – exp6a.c

```
/* Exp 6(a): Dijkstra's Algorithm */
#include <stdio.h>
#define INF 999
int main() {
    int n, i, v, u = 0, src, min, cost[10][10], dist[10], vis[10] = {0};
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter cost matrix (999 = no edge):\n");
    for (i = 0; i < n; i++)
        for (v = 0; v < n; v++)
            scanf("%d", &cost[i][v]);
    printf("Enter source vertex: ");
    scanf("%d", &src);
    for (i = 0; i < n; i++)
        dist[i] = cost[src][i];
    dist[src] = 0;
    vis[src] = 1;
    for (i = 1; i < n; i++) {
        min = INF;
        for (v = 0; v < n; v++)               /* pick nearest unvisited */
            if (!vis[v] && dist[v] < min) { min = dist[v]; u = v; }
        if (min == INF) break;
        vis[u] = 1;
        for (v = 0; v < n; v++)               /* relax neighbours of u */
            if (!vis[v] && dist[u] + cost[u][v] < dist[v])
                dist[v] = dist[u] + cost[u][v];
    }
    printf("\nShortest distances from vertex %d:\n", src);
    for (i = 0; i < n; i++)
        printf("Vertex %d : %d\n", i, dist[i]);
    return 0;
}
```

CMD Execution

```
C:\DSA>gcc exp6a.c -o exp6a.exe

C:\DSA>exp6a.exe
Enter number of vertices: 4
Enter cost matrix (999 = no edge):
0 4 1 999
4 0 2 5
1 2 0 8
999 5 8 0
Enter source vertex: 0

Shortest distances from vertex 0:
Vertex 0 : 0
Vertex 1 : 3
Vertex 2 : 1
Vertex 3 : 8

C:\DSA>
```

You type: `4`, then the 4 matrix rows, then `0`.

Sample output to memorize: `0 3 1 8`. The direct edge 0→1 costs 4, but going 0→2→1 costs 1+2 = 3. The direct edge 0→3 does not exist, so the path is 0→2→1→3 = 1+2+5 = 8.

Main logic: This is a greedy algorithm. Each round it takes the closest vertex that is not yet finalised and fixes its distance. It then "relaxes" that vertex's neighbours: if passing through `u` makes a neighbour cheaper to reach, the neighbour's distance is updated.

Memory shortcut: "PICK – MARK – RELAX" (pick the minimum, mark it visited, relax its neighbours). Formula: `dist[u] + cost[u][v] < dist[v]`.

Viva

1. *Q: What is the time complexity?* **A:** $O(V^2)$ with an adjacency matrix. With a min-heap and adjacency list it is O((V+E) log V).
2. *Q: Does Dijkstra work with negative edge weights?* **A:** No. Once a vertex is finalised its distance is never changed, so a later negative edge could make the result wrong. Use Bellman-Ford for negative weights.
3. *Q: Is Dijkstra greedy or dynamic programming?* **A:** Greedy. It always picks the nearest unvisited vertex.

6(b) Floyd–Warshall Algorithm – All Pairs Shortest Path

Aim: To find the shortest distances between every pair of vertices using the Floyd–Warshall algorithm.

Algorithm

1. Read `n` and the cost matrix `d` (999 = no edge, 0 on the diagonal).
2. For each intermediate vertex `k = 0 … n-1`:
3. For each source `i`:
4. For each destination `j`:
5. If `d[i][k] + d[k][j] < d[i][j]`, set `d[i][j] = d[i][k] + d[k][j]`.
6. Print the final matrix `d`. Stop.

Program – exp6b.c

```
/* Exp 6(b): Floyd-Warshall Algorithm */
#include <stdio.h>
int main() {
    int n, i, j, k, d[10][10];
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter cost matrix (999 = no edge):\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &d[i][j]);
    for (k = 0; k < n; k++)                    /* via every vertex k */
        for (i = 0; i < n; i++)
            for (j = 0; j < n; j++)
                if (d[i][k] + d[k][j] < d[i][j])
                    d[i][j] = d[i][k] + d[k][j];
    printf("\nAll-pairs shortest path matrix:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++)
            printf("%d\t", d[i][j]);
        printf("\n");
    }
    return 0;
}
```

CMD Execution

```
C:\DSA>gcc exp6b.c -o exp6b.exe

C:\DSA>exp6b.exe
Enter number of vertices: 4
Enter cost matrix (999 = no edge):
0 4 1 999
4 0 2 5
1 2 0 8
999 5 8 0

All-pairs shortest path matrix:
0    3    1    8
3    0    2    5
1    2    0    7
8    5    7    0

C:\DSA>
```

You type: `4`, then the 4 matrix rows (the same graph G4 as 6a).

Sample output to memorize:

```
0 3 1 8
3 0 2 5
1 2 0 7
8 5 7 0
```

Row 0 is exactly Dijkstra's answer (`0 3 1 8`), which is a good self-check. The entry 2→3 changes from 8 to 7 (2→1→3 = 2+5).

Main logic: Dynamic programming. After step `k`, `d[i][j]` holds the shortest path from i to j that uses only vertices `0…k` as stopovers. Three nested loops, and **k must be the outermost loop**.

Memory shortcut: "K-I-J, go via K". Formula: `d[i][j] = min(d[i][j], d[i][k] + d[k][j])`.

Viva

1. *Q: What is the time and space complexity?* **A:** $O(V^3)$ time and $O(V^2)$ space.
2. *Q: Why must `k` be the outermost loop?* **A:** Paths through vertex `k` must be built only after all paths using vertices `0…k-1` are complete. That ordering is the DP property.
3. *Q: How do you detect a negative cycle?* **A:** After the algorithm finishes, if any diagonal entry `d[i][i]` is less than 0, a negative cycle exists.

EXPERIMENT 7 – Minimum Spanning Tree

7(a) Kruskal's Algorithm

Aim: To find the minimum cost spanning tree of a graph using Kruskal's algorithm.

Algorithm

1. Read `n` and the cost matrix. Replace 0 (no edge) with 999.
2. Set `parent[i] = i` for every vertex (each vertex is its own set).
3. While fewer than `n-1` edges are selected:
4. Find the smallest remaining edge `(a, b)`.
5. Find the roots: `u = find(a)` and `v = find(b)`.
6. If `u != v` (no cycle), accept the edge, add its cost, and set `parent[v] = u`.
7. Delete the edge (set it to 999). After the loop, print the total cost.

Program – exp7a.c

```
/* Exp 7(a): Kruskal's Algorithm */
#include <stdio.h>
int parent[10];
int find(int i) {
    while (parent[i] != i)
        i = parent[i];
    return i;
}
int main() {
    int n, i, j, a = 0, b = 0, u, v, min, ne = 1, total = 0, cost[10][10];
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter cost matrix (0 = no edge):\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0) cost[i][j] = 999;
        }
    for (i = 0; i < n; i++)
        parent[i] = i;
    printf("\nEdges in MST:\n");
    while (ne < n) {
        min = 999;
        for (i = 0; i < n; i++)               /* smallest remaining edge */
            for (j = 0; j < n; j++)
                if (cost[i][j] < min) { min = cost[i][j]; a = i; b = j; }
        if (min == 999) break;
        u = find(a);
        v = find(b);
        if (u != v) {                         /* no cycle -> accept */
            printf("Edge %d: (%d - %d) cost = %d\n", ne++, a, b, min);
            total += min;
            parent[v] = u;
        }
        cost[a][b] = cost[b][a] = 999;
    }
    printf("Minimum cost = %d\n", total);
    return 0;
}
```

CMD Execution

```
C:\DSA>gcc exp7a.c -o exp7a.exe

C:\DSA>exp7a.exe
Enter number of vertices: 4
Enter cost matrix (0 = no edge):
0 2 3 0
2 0 1 4
3 1 0 5
0 4 5 0

Edges in MST:
Edge 1: (1 - 2) cost = 1
Edge 2: (0 - 1) cost = 2
Edge 3: (1 - 3) cost = 4
Minimum cost = 7

C:\DSA>
```

You type: `4`, then the 4 matrix rows.

Sample output to memorize: edges `1-2 (1)`, `0-1 (2)`, `1-3 (4)`, **total 7**. Edge 0-2 (cost 3) is rejected because it would form a cycle.

Main logic: Sort-by-weight greedy, done here by repeatedly picking the minimum edge. Union-find (`parent[]`) checks whether both ends of an edge are already connected. If they are, the edge is skipped because it would form a cycle.

Memory shortcut: "SMALLEST EDGE, NO CYCLE". Kruskal = **E**dge-based (K for Kruskal, think "edges").

Viva

1. *Q: What is the time complexity?* **A:** O(E log E), dominated by sorting the edges. This matrix version is $O(V^3)$.
2. *Q: How does Kruskal detect a cycle?* **A:** With union-find (disjoint sets). If `find(a) == find(b)`, the two vertices are already connected, so adding the edge would make a cycle.
3. *Q: How many edges does an MST have?* **A:** Exactly $V - 1$ edges.

7(b) Prim's Algorithm

Aim: To find the minimum cost spanning tree of a graph using Prim's algorithm.

Algorithm

1. Read `n` and the cost matrix. Replace 0 (no edge) with 999.
2. Mark vertex 0 as visited (it is the start of the tree).
3. Repeat `n-1` times:
4. Among all edges `(i, j)` with `i` visited and `j` unvisited, pick the minimum.
5. Mark `j` visited, print the edge, and add its cost to the total.
6. Print the total cost. Stop.

Program – exp7b.c

```
/* Exp 7(b): Prim's Algorithm */
#include <stdio.h>
int main() {
    int n, i, j, e, a = 0, b = 0, min, total = 0, cost[10][10], vis[10] = {0};
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter cost matrix (0 = no edge):\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0) cost[i][j] = 999;
        }
    vis[0] = 1;                                 /* start from vertex 0 */
    printf("\nEdges in MST:\n");
    for (e = 1; e < n; e++) {
        min = 999;
        for (i = 0; i < n; i++)                 /* cheapest edge tree -> outside */
            if (vis[i])
                for (j = 0; j < n; j++)
                    if (!vis[j] && cost[i][j] < min) { min = cost[i][j]; a = i; b = j; }
        if (min == 999) break;
        vis[b] = 1;
        printf("Edge %d: (%d - %d) cost = %d\n", e, a, b, min);
        total += min;
    }
    printf("Minimum cost = %d\n", total);
    return 0;
}
```

CMD Execution

```
C:\DSA>gcc exp7b.c -o exp7b.exe

C:\DSA>exp7b.exe
Enter number of vertices: 4
Enter cost matrix (0 = no edge):
0 2 3 0
2 0 1 4
3 1 0 5
0 4 5 0

Edges in MST:
Edge 1: (0 - 1) cost = 2
Edge 2: (1 - 2) cost = 1
Edge 3: (1 - 3) cost = 4
Minimum cost = 7

C:\DSA>
```

You type: `4`, then the 4 matrix rows (the same graph M4 as 7a).

Sample output to memorize: edges `0-1 (2)`, `1-2 (1)`, `1-3 (4)`, **total 7**. This is the same total as Kruskal but the edges come out in a different order, which is a classic viva point.

Main logic: The tree grows outward from vertex 0. Each step adds the cheapest edge that joins a vertex inside the tree to a vertex outside it. A cycle is impossible because the new vertex is always outside the tree.

Memory shortcut: "GROW THE TREE FROM 0". Prim = **P**oint/vertex-based.

Viva

1. *Q: What is the difference between Prim and Kruskal?* **A:** Prim grows a single tree vertex by vertex and is better for dense graphs. Kruskal picks edges globally and may form a forest first, which suits sparse graphs.
2. *Q: What is the time complexity?* **A:** $O(V^2)$ with an adjacency matrix, or $O(E \log V)$ with a heap.
3. *Q: Is the MST unique?* **A:** It is unique if all edge weights are distinct. Otherwise the edges may differ, but the total cost is always the same.

EXPERIMENT 8 – Graph Traversal & Topological Sort

8(a) Depth First Search (DFS)

Aim: To traverse a graph using Depth First Search.

Algorithm

1. Read `n` and the adjacency matrix.
2. Read the starting vertex `s`.
3. Call `dfs(s)`.
4. In `dfs(v)`: mark `v` visited and print it.
5. For each vertex `i`: if `a[v][i] == 1` and `i` is unvisited, call `dfs(i)` recursively.
6. Stop when the recursion returns.

Program – exp8a.c

```
/* Exp 8(a): Depth First Search */
#include <stdio.h>
int a[10][10], vis[10], n;
void dfs(int v) {
    int i;
    vis[v] = 1;
    printf("%d ", v);
    for (i = 0; i < n; i++)
        if (a[v][i] && !vis[i])
            dfs(i);
}
int main() {
    int i, j, s;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &a[i][j]);
    printf("Enter starting vertex: ");
    scanf("%d", &s);
    printf("\nDFS traversal: ");
    dfs(s);
    printf("\n");
    return 0;
}
```

CMD Execution

```
C:\DSA>gcc exp8a.c -o exp8a.exe

C:\DSA>exp8a.exe
Enter number of vertices: 5
Enter adjacency matrix:
0 1 1 0 0
1 0 0 1 0
1 0 0 0 1
0 1 0 0 0
0 0 1 0 0
Enter starting vertex: 0

DFS traversal: 0 1 3 2 4

C:\DSA>
```

You type: `5`, then the 5 matrix rows, then `0`. Graph edges: 0-1, 0-2, 1-3, 2-4.

Sample output to memorize: `0 1 3 2 4`. DFS goes deep along 0→1→3, backtracks, then visits 2→4.

Main logic: Recursion acts as a stack. DFS goes as deep as possible, then backtracks. `vis[]` prevents visiting a vertex twice.

Memory shortcut: "DFS = Dive deep, uses Stack (recursion)".

Viva

1. *Q: Which data structure does DFS use?* **A:** A stack. Here it is the recursion call stack.
2. *Q: What is the time complexity?* **A:** $O(V^2)$ with an adjacency matrix, or O(V+E) with an adjacency list.
3. *Q: What are the applications of DFS?* **A:** Cycle detection, topological sort, connected components, path finding, and maze solving.

8(b) Breadth First Search (BFS)

Aim: To traverse a graph using Breadth First Search.

Algorithm

1. Read `n`, the adjacency matrix, and the starting vertex `s`.
2. Enqueue `s` and mark it visited.
3. While the queue is not empty:
4. Dequeue a vertex `v` and print it.
5. Enqueue every unvisited neighbour of `v` and mark it visited.
6. Stop when the queue is empty.

Program – exp8b.c

```
/* Exp 8(b): Breadth First Search */
#include <stdio.h>
int main() {
    int n, i, j, s, v, a[10][10], vis[10] = {0}, q[10], f = 0, r = 0;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &a[i][j]);
    printf("Enter starting vertex: ");
    scanf("%d", &s);
    q[r++] = s;                             /* enqueue start */
    vis[s] = 1;
    printf("\nBFS traversal: ");
    while (f < r) {
        v = q[f++];                         /* dequeue */
        printf("%d ", v);
        for (i = 0; i < n; i++)
            if (a[v][i] && !vis[i]) { q[r++] = i; vis[i] = 1; }
    }
    printf("\n");
    return 0;
}
```

CMD Execution

```
C:\DSA>gcc exp8b.c -o exp8b.exe

C:\DSA>exp8b.exe
Enter number of vertices: 5
Enter adjacency matrix:
0 1 1 0 0
1 0 0 1 0
1 0 0 0 1
0 1 0 0 0
0 0 1 0 0
Enter starting vertex: 0

BFS traversal: 0 1 2 3 4

C:\DSA>
```

You type: `5`, then the 5 matrix rows, then `0` (the same graph U5 as 8a).

Sample output to memorize: `0 1 2 3 4`. Level 0 is {0}, level 1 is {1, 2}, level 2 is {3, 4}. Compare with DFS `0 1 3 2 4`.

Main logic: BFS visits level by level using a FIFO queue (`q[]` with front `f` and rear `r`). A vertex is marked visited when it is **enqueued**, so it never enters the queue twice.

Memory shortcut: "BFS = Broad, level by level, uses Queue". Same graph: DFS **0 1 3 2 4**, BFS **0 1 2 3 4**.

Viva

1. *Q: Which data structure does BFS use?* **A:** A queue (FIFO).
2. *Q: Why use BFS for shortest paths?* **A:** In an unweighted graph, BFS reaches each vertex using the fewest possible edges.
3. *Q: What is the time complexity?* **A:** $O(V^2)$ with an adjacency matrix, or O(V+E) with an adjacency list.

8(c) Topological Sorting

Aim: To find a topological ordering of the vertices of a Directed Acyclic Graph (DAG).

Algorithm

1. Read `n` and the directed adjacency matrix.
2. Compute the indegree of each vertex (the sum of its column).
3. Repeat `n` times:
4. Find an unvisited vertex `i` with indegree 0. If none exists, the graph has a cycle, so stop.
5. Mark `i` visited and print it.
6. For every edge `i → j`, decrease the indegree of `j`.
7. Stop.

Program – exp8c.c

```
/* Exp 8(c): Topological Sorting (Kahn's / indegree method) */
#include <stdio.h>
int main() {
    int n, i, j, k, a[10][10], in[10] = {0}, vis[10] = {0};
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix (directed):\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
            in[j] += a[i][j];                /* indegree of column j */
        }
    printf("\nTopological order: ");
    for (k = 0; k < n; k++) {
        for (i = 0; i < n; i++)              /* find vertex with indegree 0 */
            if (in[i] == 0 && !vis[i]) break;
        if (i == n) { printf("\nCycle exists! Not a DAG."); break; }
        vis[i] = 1;
        printf("%d ", i);
        for (j = 0; j < n; j++)              /* remove its outgoing edges */
            if (a[i][j]) in[j]--;
    }
    printf("\n");
    return 0;
}
```

CMD Execution

```
C:\DSA>gcc exp8c.c -o exp8c.exe

C:\DSA>exp8c.exe
Enter number of vertices: 4
Enter adjacency matrix (directed):
0 0 0 1
1 0 1 0
0 0 0 1
0 0 0 0

Topological order: 1 0 2 3

C:\DSA>
```

You type: `4`, then the 4 matrix rows. Graph edges: 1→0, 1→2, 0→3, 2→3. The starting indegrees are 0:1, 1:0, 2:1, 3:2.

Sample output to memorize: `1 0 2 3`. Vertex 1 has no incoming edge, so it comes first. Vertex 3 depends on both 0 and 2, so it comes last.

Main logic: Kahn's algorithm. Repeatedly output a vertex that no remaining vertex points to (indegree 0), then remove its outgoing edges. If no vertex has indegree 0 while vertices remain, the graph has a cycle.

Memory shortcut: "ZERO-IN goes OUT": the vertex with zero indegree is output first.

Viva

1. *Q: For which graphs is topological sort possible?* **A:** Only for Directed Acyclic Graphs (DAGs).
2. *Q: Is the topological order unique?* **A:** Not always. Any order where every edge u→v has u before v is valid.
3. *Q: What are the applications?* **A:** Course prerequisite ordering, task scheduling, build systems (makefiles), and compiler dependency resolution.

EXPERIMENT 9 – Sorting & Hashing

> All four sorting programs use the **same input** `5 3 4 1 2` and print `1 2 3 4 5`. Learn the input once.

9(a) Insertion Sort

Aim: To sort a list of elements using insertion sort.

Algorithm

1. Read `n` and the array `a[]`.
2. For `i = 1` to `n-1`:
3. Set `key = a[i]` and `j = i-1`.
4. While `j >= 0` and `a[j] > key`: shift `a[j]` to `a[j+1]` and decrease `j`.
5. Place `key` at `a[j+1]`.
6. Print the array. Stop.

Program – exp9a.c

```
/* Exp 9(a): Insertion Sort */
#include <stdio.h>
int main() {
    int n, i, j, key, a[20];
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    for (i = 1; i < n; i++) {
        key = a[i];
        for (j = i - 1; j >= 0 && a[j] > key; j--)
            a[j + 1] = a[j];                  /* shift bigger ones right */
        a[j + 1] = key;
    }
    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

CMD Execution

```
C:\DSA>gcc exp9a.c -o exp9a.exe

C:\DSA>exp9a.exe
Enter number of elements: 5
Enter 5 elements: 5 3 4 1 2
Sorted array: 1 2 3 4 5

C:\DSA>
```

You type: `5`, then `5 3 4 1 2`.

Passes (for explanation): `3 5 4 1 2` → `3 4 5 1 2` → `1 3 4 5 2` → `1 2 3 4 5`

Main logic: This works like sorting playing cards in your hand. Take the next card (`key`), shift every bigger card one place right, and drop the card into the gap.

Memory shortcut: "KEY – SHIFT – DROP".

Viva

1. *Q: What are the best and worst cases?* **A:** Best case O(n), when the array is already sorted. Worst case O(n^2), when it is reverse sorted.
2. *Q: Is insertion sort stable and in-place?* **A:** Yes, it is both stable and in-place.
3. *Q: When is insertion sort preferred?* **A:** For small arrays or arrays that are nearly sorted. It is also adaptive.

9(b) Selection Sort

Aim: To sort a list of elements using selection sort.

Algorithm

1. Read `n` and the array `a[]`.
2. For `i = 0` to `n-2`:
3. Set `min = i`.
4. For `j = i+1` to `n-1`: if `a[j] < a[min]`, set `min = j`.
5. Swap `a[i]` and `a[min]`.
6. Print the array. Stop.

Program – exp9b.c

```
/* Exp 9(b): Selection Sort */
#include <stdio.h>
int main() {
    int n, i, j, min, t, a[20];
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    for (i = 0; i < n - 1; i++) {
        min = i;
        for (j = i + 1; j < n; j++)           /* find smallest in rest */
            if (a[j] < a[min]) min = j;
        t = a[i]; a[i] = a[min]; a[min] = t;
    }
    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

CMD Execution

```
C:\DSA>gcc exp9b.c -o exp9b.exe

C:\DSA>exp9b.exe
Enter number of elements: 5
Enter 5 elements: 5 3 4 1 2
Sorted array: 1 2 3 4 5

C:\DSA>
```

You type: `5`, then `5 3 4 1 2`.

Passes: `1 3 4 5 2` → `1 2 4 5 3` → `1 2 3 5 4` → `1 2 3 4 5`

Main logic: On each pass, select the smallest element of the unsorted part and swap it to the front. The sorted part grows by one element per pass.

Memory shortcut: "FIND MIN – SWAP TO FRONT".

Viva

1. *Q: What is the time complexity?* **A:** O(n^2) in every case, including best, average and worst.
2. *Q: How many swaps does it do?* **A:** At most n − 1 swaps. This is useful when writing to memory is expensive.
3. *Q: Is selection sort stable?* **A:** No, the standard version is not stable. A long-distance swap can reorder equal elements.

9(c) Quick Sort

Aim: To sort a list of elements using quick sort (divide and conquer).

Algorithm

1. Read `n` and the array `a[]`, then call `quick(a, 0, n-1)`.
2. In `quick(lo, hi)`: if `lo < hi`, partition the array and get pivot index `p`.
3. Partition: set `pivot = a[hi]` and `i = lo-1`.
4. For `j = lo` to `hi-1`: if `a[j] < pivot`, swap `a[++i]` with `a[j]`.
5. Swap `a[i+1]` with `a[hi]` and return `i+1`.
6. Recursively call `quick(lo, p-1)` and `quick(p+1, hi)`.
7. Print the array. Stop.

Program – exp9c.c

```
/* Exp 9(c): Quick Sort */
#include <stdio.h>
void swap(int *x, int *y) { int t = *x; *x = *y; *y = t; }
int partition(int a[], int lo, int hi) {
    int p = a[hi], i = lo - 1, j;             /* last element = pivot */
    for (j = lo; j < hi; j++)
        if (a[j] < p) swap(&a[++i], &a[j]);
    swap(&a[i + 1], &a[hi]);
    return i + 1;
}
void quick(int a[], int lo, int hi) {
    if (lo < hi) {
        int p = partition(a, lo, hi);
        quick(a, lo, p - 1);
        quick(a, p + 1, hi);
    }
}
int main() {
    int n, i, a[20];
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    quick(a, 0, n - 1);
    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

CMD Execution

```
C:\DSA>gcc exp9c.c -o exp9c.exe

C:\DSA>exp9c.exe
Enter number of elements: 5
Enter 5 elements: 5 3 4 1 2
Sorted array: 1 2 3 4 5

C:\DSA>
```

You type: `5`, then `5 3 4 1 2`.

First partition (pivot = 2): `1 2 4 5 3`. The pivot 2 is now at index 1, its final position.

Main logic: Choose a pivot (here the last element). Partition the array so that smaller elements go left and larger elements go right, putting the pivot in its final place. Then sort both sides recursively. This is the Lomuto partition scheme.

Memory shortcut: "PIVOT LAST, SMALL GOES LEFT, RECURSE BOTH".

Viva

1. *Q: What is the time complexity?* **A:** Best and average case O(n log n). Worst case O(n^2), which happens when the array is already sorted and the last element is the pivot.
2. *Q: How can the worst case be avoided?* **A:** Use a random pivot or the median-of-three pivot.
3. *Q: Is quick sort stable?* **A:** No. It is in-place, using only O(log n) recursion stack.

9(d) Merge Sort

Aim: To sort a list of elements using merge sort (divide and conquer).

Algorithm

1. Read `n` and the array `a[]`, then call `msort(a, 0, n-1)`.
2. In `msort(lo, hi)`: if `lo < hi`, set `mid = (lo+hi)/2`.
3. Recursively sort the left half `msort(lo, mid)` and the right half `msort(mid+1, hi)`.
4. Merge: compare the front elements of both halves and copy the smaller one into `t[]`.
5. Copy any leftover elements, then copy `t[]` back into `a[lo..hi]`.
6. Print the array. Stop.

Program – exp9d.c

```
/* Exp 9(d): Merge Sort */
#include <stdio.h>
void merge(int a[], int lo, int mid, int hi) {
    int t[20], i = lo, j = mid + 1, k = 0;
    while (i <= mid && j <= hi)
        t[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    while (i <= mid) t[k++] = a[i++];
    while (j <= hi)  t[k++] = a[j++];
    for (i = lo, k = 0; i <= hi; i++, k++)
        a[i] = t[k];
}
void msort(int a[], int lo, int hi) {
    if (lo < hi) {
        int mid = (lo + hi) / 2;
        msort(a, lo, mid);
        msort(a, mid + 1, hi);
        merge(a, lo, mid, hi);
    }
}
int main() {
    int n, i, a[20];
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    msort(a, 0, n - 1);
    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

CMD Execution

```
C:\DSA>gcc exp9d.c -o exp9d.exe

C:\DSA>exp9d.exe
Enter number of elements: 5
Enter 5 elements: 5 3 4 1 2
Sorted array: 1 2 3 4 5

C:\DSA>
```

You type: `5`, then `5 3 4 1 2`.

Split/merge: `[5 3 4] [1 2]` → `[3 4 5] [1 2]` → `1 2 3 4 5`

Main logic: Split the array in half until each piece has one element, which is already sorted. Then merge sorted pieces back together by repeatedly taking the smaller front element.

Memory shortcut: "SPLIT – SORT – MERGE" (divide, conquer, combine).

Viva

1. *Q: What is the time complexity?* **A:** O(n log n) in all cases, using the recurrence T(n) = 2T(n/2) + n.
2. *Q: What is the space complexity?* **A:** O(n) extra space for the temporary array, so it is not in-place.
3. *Q: Is merge sort stable?* **A:** Yes, because it uses `<=` when merging. It is preferred for linked lists and external sorting.

9(e) Hash Table using Linear Probing

Aim: To implement a hash table using open addressing with linear probing.

Algorithm

1. Create a table of size 10 and set every slot to −1 (empty).
2. Read `n` keys. For each key, try `i = 0, 1, 2, …`.
3. Compute `idx = (key % 10 + i) % 10`.
4. If `h[idx]` is empty, store the key there and stop probing. If all 10 slots are tried, report that the table is full.
5. Display the table.
6. Search: probe the same sequence until the key is found (found) or an empty slot is reached (not found).

Program – exp9e.c

```
/* Exp 9(e): Hash Table using Linear Probing */
#include <stdio.h>
#define SIZE 10
int main() {
    int h[SIZE], n, i, k, key, idx;
    for (i = 0; i < SIZE; i++) h[i] = -1;     /* -1 = empty slot */
    printf("Enter number of keys: ");
    scanf("%d", &n);
    printf("Enter %d keys: ", n);
    for (k = 0; k < n; k++) {
        scanf("%d", &key);
        for (i = 0; i < SIZE; i++) {
            idx = (key % SIZE + i) % SIZE;    /* h(k) + i */
            if (h[idx] == -1) { h[idx] = key; break; }
        }
        if (i == SIZE) printf("Table full, %d not inserted\n", key);
    }
    printf("\nHash Table:\n");
    for (i = 0; i < SIZE; i++) {
        if (h[i] == -1) printf("%d : -\n", i);
        else printf("%d : %d\n", i, h[i]);
    }
    printf("Enter key to search: ");
    scanf("%d", &key);
    for (i = 0; i < SIZE; i++) {
        idx = (key % SIZE + i) % SIZE;
        if (h[idx] == key) { printf("Key %d found at index %d\n", key, idx); return 0; }
        if (h[idx] == -1) break;
    }
    printf("Key %d not found\n", key);
    return 0;
}
```

CMD Execution

```
C:\DSA>gcc exp9e.c -o exp9e.exe

C:\DSA>exp9e.exe
Enter number of keys: 5
Enter 5 keys: 23 43 13 27 37

Hash Table:
0 : -
1 : -
2 : -
3 : 23
4 : 43
5 : 13
6 : -
7 : 27
8 : 37
9 : -
Enter key to search: 13
Key 13 found at index 5

C:\DSA>
```

You type: `5`, then `23 43 13 27 37`, then `13`.

Trace: 23→3. 43→3 is taken, so 4. 13→3 and 4 are taken, so 5. 27→7. 37→7 is taken, so 8.

Main logic: The hash function is `key % 10`. On a collision, move to the **next** slot (+1, +2, +3 …) and wrap around using `% SIZE`.

Memory shortcut: "COLLIDE → STEP +1". Formula: `(h + i) % 10`.

Viva

1. *Q: What is a collision?* **A:** A collision happens when two keys hash to the same index (for example, 23 and 43 both give 3).
2. *Q: What is the drawback of linear probing?* **A:** Primary clustering. Occupied slots form long runs, which makes later insertions and searches slower.
3. *Q: What is the load factor?* **A:** α = n / table size. Here it is 5/10 = 0.5. Performance drops as α approaches 1.

9(f) Hash Table using Quadratic Probing

Aim: To implement a hash table using open addressing with quadratic probing.

Algorithm

1. Create a table of size 10 and set every slot to −1 (empty).
2. Read `n` keys. For each key, try `i = 0, 1, 2, …`.
3. Compute `idx = (key % 10 + i*i) % 10`.
4. If `h[idx]` is empty, store the key there and stop. If no slot is found after 10 tries, report it.
5. Display the table.
6. Search using the same `i*i` sequence until the key is found or an empty slot is reached.

Program – exp9f.c

```
/* Exp 9(f): Hash Table using Quadratic Probing */
#include <stdio.h>
#define SIZE 10
int main() {
    int h[SIZE], n, i, k, key, idx;
    for (i = 0; i < SIZE; i++) h[i] = -1;     /* -1 = empty slot */
    printf("Enter number of keys: ");
    scanf("%d", &n);
    printf("Enter %d keys: ", n);
    for (k = 0; k < n; k++) {
        scanf("%d", &key);
        for (i = 0; i < SIZE; i++) {
            idx = (key % SIZE + i * i) % SIZE; /* h(k) + i*i */
            if (h[idx] == -1) { h[idx] = key; break; }
        }
        if (i == SIZE) printf("No free slot, %d not inserted\n", key);
    }
    printf("\nHash Table:\n");
    for (i = 0; i < SIZE; i++) {
        if (h[i] == -1) printf("%d : -\n", i);
        else printf("%d : %d\n", i, h[i]);
    }
    printf("Enter key to search: ");
    scanf("%d", &key);
    for (i = 0; i < SIZE; i++) {
        idx = (key % SIZE + i * i) % SIZE;
        if (h[idx] == key) { printf("Key %d found at index %d\n", key, idx); return 0; }
        if (h[idx] == -1) break;
    }
    printf("Key %d not found\n", key);
    return 0;
}
```

CMD Execution

```
C:\DSA>gcc exp9f.c -o exp9f.exe

C:\DSA>exp9f.exe
Enter number of keys: 5
Enter 5 keys: 23 43 13 27 37

Hash Table:
0 : -
1 : 37
2 : -
3 : 23
4 : 43
5 : -
6 : -
7 : 13
8 : 27
9 : -
Enter key to search: 37
Key 37 found at index 1

C:\DSA>
```

You type: `5`, then `23 43 13 27 37`, then `37`.

Trace: 23→3. 43→3+1 = 4. 13→3, then 3+1 = 4, both taken, then 3+4 = 7. 27→7 is taken, so 7+1 = 8. 37→7 and 8 are taken, then (7+4)%10 = **1**.

Main logic: This is the same as linear probing, but the step is `i²` (+1, +4, +9, +16 ...). Jumping further spreads keys out and avoids long clusters.

Memory shortcut: "COLLIDE → JUMP i²". Formula: `(h + i*i) % 10`. Same keys: Linear puts 37 at **8**, Quadratic puts 37 at **1**.

Viva

1. *Q: What advantage does quadratic probing have over linear probing?* **A:** It reduces primary clustering because the probe positions spread out.
2. *Q: What is secondary clustering?* **A:** Keys with the same home index still follow the same probe sequence. Double hashing removes this.
3. *Q: Can quadratic probing fail even when the table has free slots?* **A:** Yes. It may not visit every slot. If the table size is prime and the load factor $\alpha \leq 0.5$, a free slot is guaranteed to be found.

CMD EXECUTION CHEAT SHEET (All 12)

```
C:\>cd C:\DSA
C:\DSA>gcc exp6a.c -o exp6a.exe   &   exp6a.exe
```

Prog	Compile & Run	Input you type	Output to expect
6a Dijkstra	`gcc exp6a.c -o exp6a.exe` → `exp6a.exe`	`4` / G4 / src `0`	`0 3 1 8`
6b Floyd	`gcc exp6b.c -o exp6b.exe` → `exp6b.exe`	`4` / G4	rows `0 3 1 8`, `3 0 2 5`, `1 2 0 7`, `8 5 7 0`
7a Kruskal	`gcc exp7a.c -o exp7a.exe` → `exp7a.exe`	`4` / M4	`(1-2)1, (0-1)2, (1-3)4`, cost **7**
7b Prim	`gcc exp7b.c -o exp7b.exe` → `exp7b.exe`	`4` / M4	`(0-1)2, (1-2)1, (1-3)4`, cost **7**
8a DFS	`gcc exp8a.c -o exp8a.exe` → `exp8a.exe`	`5` / U5 / `0`	`0 1 3 2 4`
8b BFS	`gcc exp8b.c -o exp8b.exe` → `exp8b.exe`	`5` / U5 / `0`	`0 1 2 3 4`
8c Topo	`gcc exp8c.c -o exp8c.exe` → `exp8c.exe`	`4` / `0001 1010 0001 0000`	`1 0 2 3`
9a Insertion	`gcc exp9a.c -o exp9a.exe` → `exp9a.exe`	`5` / `5 3 4 1 2`	`1 2 3 4 5`
9b Selection	`gcc exp9b.c -o exp9b.exe` → `exp9b.exe`	`5` / `5 3 4 1 2`	`1 2 3 4 5`
9c Quick	`gcc exp9c.c -o exp9c.exe` → `exp9c.exe`	`5` / `5 3 4 1 2`	`1 2 3 4 5`
9d Merge	`gcc exp9d.c -o exp9d.exe` → `exp9d.exe`	`5` / `5 3 4 1 2`	`1 2 3 4 5`
9e Linear	`gcc exp9e.c -o exp9e.exe` → `exp9e.exe`	`5` / `23 43 13 27 37` / `13`	slots 3,4,5,7,8 · found at **5**
9f Quadratic	`gcc exp9f.c -o exp9f.exe` → `exp9f.exe`	`5` / `23 43 13 27 37` / `37`	slots 3,4,7,8,1 · found at **1**

Input matrices

```
G4 (6a,6b)      M4 (7a,7b)    U5 (8a,8b)    DAG (8c)
0 4 1 999       0 2 3 0       0 1 1 0 0     0 0 0 1
4 0 2 5         2 0 1 4       1 0 0 1 0     1 0 1 0
1 2 0 8         3 1 0 5       1 0 0 0 1     0 0 0 1
999 5 8 0       0 4 5 0       0 1 0 0 0     0 0 0 0
                              0 0 1 0 0
```

One-line memory hooks

Prog	Hook	Key line
Dijkstra	PICK–MARK–RELAX	`dist[u]+cost[u][v] < dist[v]`
Floyd	K-I-J via K	`d[i][k]+d[k][j] < d[i][j]`
Kruskal	Smallest edge, no cycle	`if(find(a)!=find(b))`
Prim	Grow tree from 0	`if(vis[i] && !vis[j])`
DFS	Dive deep, stack	`if(a[v][i]&&!vis[i]) dfs(i);`
BFS	Level by level, queue	`v=q[f++]; ... q[r++]=i;`
Topo	Zero-in goes out	`if(in[i]==0&&!vis[i])`
Insertion	Key–shift–drop	`a[j+1]=a[j]`
Selection	Find min, swap front	`if(a[j]<a[min]) min=j;`
Quick	Pivot last, small left	`if(a[j]<p) swap(&a[++i],&a[j]);`
Merge	Split–sort–merge	`t[k++]=(a[i]<=a[j])?a[i++]:a[j++];`
Linear	Collide → +1	`(key%SIZE + i)%SIZE`
Quadratic	Collide → +i²	`(key%SIZE + i*i)%SIZE`

Complexity table (quick viva revision)

Algorithm	Time	Space
Dijkstra (matrix)	$O(V^2)$	O(V)
Floyd–Warshall	$O(V^3)$	$O(V^2)$
Kruskal	O(E log E)	O(V)
Prim (matrix)	$O(V^2)$	O(V)
DFS / BFS (matrix)	$O(V^2)$	O(V)
Topological sort	$O(V^2)$ matrix / O(V+E) list	O(V)
Insertion	O(n) best, $O(n^2)$ worst	O(1)
Selection	$O(n^2)$ all cases	O(1)
Quick	O(n log n) avg, $O(n^2)$ worst	O(log n)
Merge	O(n log n) all cases	O(n)
Hashing (open addressing)	O(1) avg, O(n) worst	O(table size)

Exam-hall tips

- If `gcc` is "not recognized", GCC is not on PATH. Check with `gcc --version`.
- You can type a matrix row by row, or all numbers on one line separated by spaces. `scanf` accepts both.
- Windows also accepts `exp6a` without `.exe` when you run a program.